• COURS COMPLET

JavaScript Moderne

Des fondamentaux du langage jusqu'aux fonctionnalités ES6+ : variables, fonctions, classes, modules, Promises et async/await.

17	2	100+	Débutant
chapitres	parties	exemples de code	→ avancé

PARTIE 1

Fondamentaux JavaScript

Variables, types, opérateurs, conditions, boucles, fonctions, tableaux, objets — la base indispensable.

PARTIE 2

JavaScript ES6+

Template literals, destructuration, spread/rest, classes, modules, Promises, async/await, Map & Set.

Table des matières

Un parcours en deux temps : les bases du langage, puis les fonctionnalités modernes introduites par ES6 (2015) et les versions suivantes.

PARTIE 1 FONDAMENTAUX JAVASCRIPT

01 **Introduction à JavaScript**
Qu'est-ce que JS ? Où et comment l'utiliser ?

02 **Variables & Déclarations**
var, let, const, portée de bloc, hoisting, Temporal Dead Zone

03 **Types de données**
Les 7 types primitifs, typeof, conversions

04 **Opérateurs**
Arithmétiques, comparaison, logiques, affectation

05 **Conditions**
if/else, switch, opérateur ternaire

06 **Boucles**
for, while, do...while, for...of, for...in

07 **Fonctions**
Déclaration, paramètres, fonctions fléchées, this lexical

08 **Tableaux (Arrays)**
Créer, modifier, parcourir, map / filter / reduce

09 **Objets**
Clé-valeur, méthodes, this, déstructuration de base

PARTIE 2 JAVASCRIPT ES6+ (JAVASCRIPT MODERNE)

10 **Template literals**
Interpolation, multi-lignes, tagged templates

11 **Destructuration avancée**
Tableaux, objets, valeurs par défaut, imbrication

12 **Spread & Rest**
L'opérateur ... pour étaler ou regrouper des valeurs

13 **Classes**
POO, héritage, getters/setters, champs privés

14 **Modules ES6**
import / export, organisation du code

15 **Promises & async/await**
Gérer l'asynchrone proprement

16 **Itérateurs, Map & Set**
Nouvelles collections et protocole itérable

17 **Paramètres & objets améliorés**
Raccourcis de syntaxe ES6 pour fonctions et objets

PARTIE 1 — FONDAMENTAUX

01 Introduction à JavaScript

Qu'est-ce que JS ? Où et comment l'utiliser ?

Qu'est-ce que JavaScript ?

JavaScript (abrégé JS) est un langage de programmation créé en 1995. C'est le seul langage natif des navigateurs web : il permet de rendre les pages interactives (boutons, animations, formulaires) et même de construire des applications entières (SPA, PWA…).

Côté client (browser)
S'exécute dans Chrome, Firefox, Safari…

Côté serveur (Node.js)
JS tourne aussi sur un serveur grâce à Node.js

Mobile
React Native, Ionic : JS pour les apps mobiles

Bureau
Electron : apps desktop avec JS (VS Code, Slack…)

Comment exécuter du JavaScript ?

Deux façons principales d'intégrer du JS dans une page web :

```
// méthode 1 — balise script dans le HTML
<script>
  console.log('Bonjour !');
</script>

// méthode 2 — fichier externe (recommandé)
<script src="script.js"></script>
// dans script.js
console.log('Fichier externe !');
```

Astuce — Pour ouvrir la console du navigateur, appuie sur F12 → onglet « Console ». Tout ce que tu passes à `console.log()` s'y affiche.

Ton premier programme

```
// hello world en JavaScript
console.log('Bonjour le monde !');

// afficher une alerte dans le navigateur
alert('Bienvenue en JavaScript !');

// demander une saisie à l'utilisateur
let nom = prompt('Quel est ton prénom ?');
console.log('Bonjour, ' + nom + ' !');
```

PARTIE 1 — FONDAMENTAUX

02 Variables & Déclarations

var, let, const — portée, hoisting et bonnes pratiques

Une variable est une boîte nommée qui contient une valeur, pour la réutiliser plus tard dans le programme. JavaScript propose trois mots-clés pour en déclarer : `var`, `let` et `const`.

var
Ancienne façon. Portée de fonction. À éviter en JS moderne.

let
Valeur modifiable. Portée de bloc. Usage général.

const
Valeur constante (référence). Portée de bloc. Choix par défaut.

Historique Le problème avec var

Avant ES6, `var` était le seul mot-clé disponible. Il a deux comportements problématiques : une portée de *fonction* (pas de bloc) et le *hoisting*, qui remonte virtuellement la déclaration en haut du scope — sources de bugs difficiles à repérer.

```
// var — portée de fonction, pas de bloc
function exemple() {
  if (true) {
    var x = 10; // déclaré dans le if
  }
  console.log(x); // 10 — accessible ici (bug !)
}

// var — hoisting
console.log(y); // undefined (pas d'erreur !)
var y = 5;
// équivalent à : var y; console.log(y); y = 5;
```

let La variable de bloc

`let` déclare une variable limitée au bloc `{ }` dans lequel elle est définie. Réassignable, mais pas redéclarable dans le même scope. Elle n'est pas accessible avant sa déclaration : c'est la **Temporal Dead Zone** (TDZ).

```
let compteur = 0;
compteur = 1; // OK — réassignation autorisée

if (true) {
  let x = 10;
  console.log(x); // 10
}
// console.log(x); // ReferenceError — x hors du bloc

// console.log(z); // ReferenceError (Temporal Dead Zone)
let z = 5;
```

const La constante de bloc

`const` déclare une liaison constante : la variable ne peut pas être réassignée. Attention — pour les objets et tableaux, la *référence* est constante mais le *contenu* reste mutable. C'est la déclaration à

privilégier par défaut.

```
const PI = 3.14159;
// PI = 3; // TypeError — réassignation interdite

// objet : la référence est constante, le contenu est mutable
const user = { nom: 'Alice', age: 30 };
user.age = 31; // OK — mutation de propriété
// user = {}; // TypeError

// tableau : même principe
const arr = [1, 2, 3];
arr.push(4); // OK
// arr = []; // TypeError
```

Portée des variables (scope)

```
let x = 10; // portée globale
if (true) {
  let y = 20; // portée du bloc { }
  console.log(x); // 10 (OK)
  console.log(y); // 20 (OK)
}
console.log(x); // 10 (OK)
// console.log(y); // ReferenceError : y n'existe pas ici
```

Conventions de nommage

```
// bonnes pratiques — camelCase
let prenom = 'Kiri';
let nombreDePoints = 100;
const MAX_SCORE = 999; // MAJUSCULES pour les constantes globales

// à éviter
let a = 'Kiri';        // pas descriptif
// let mon-prenom = 'X'; // tiret interdit !
// let 1valeur = 5;      // ne peut pas commencer par un chiffre
```

Attention — Les noms sont sensibles à la casse : `age`, `Age` et `AGE` sont trois variables différentes !

Règle pratique — Utilise `const` par défaut, passe à `let` uniquement quand la valeur doit changer, et ne mets plus jamais `var` dans du code moderne.

PARTIE 1 — FONDAMENTAUX

03 Types de données

Les 7 types primitifs + les objets

```
// 1. String — chaîne de caractères
let nom = 'Mamadou';
let phrase = `Bonjour ${nom}`; // template literal

// 2. Number — entier ou décimal
let age = 30;
let prix = 9.99;

// 3. Boolean — vrai ou faux
let estConnecte = true;
let estAdmin = false;

// 4. Undefined — variable non initialisée
let inconnu;
console.log(inconnu); // undefined

// 5. Null — absence intentionnelle de valeur
let vide = null;

// 6. Symbol — valeur unique (avancé)
const id = Symbol('id');

// 7. BigInt — très grands entiers
const grand = 9007199254740991n;
```

Vérifier le type avec typeof

```
console.log(typeof 'Bonjour');   // 'string'
console.log(typeof 42);          // 'number'
console.log(typeof true);        // 'boolean'
console.log(typeof undefined);   // 'undefined'
console.log(typeof null);        // 'object' ← bug historique !
console.log(typeof []);          // 'object'
console.log(typeof {});          // 'object'
```

Attention — `typeof null` retourne `'object'` alors que `null` est un primitif. C'est un bug connu depuis 1995, jamais corrigé pour ne pas casser le web existant.

Conversion de types (casting)

```
// vers un nombre
Number('42')        // 42
parseInt('42px')    // 42 (ignore le texte après le nombre)
parseFloat('3.14')  // 3.14
+'10'               // 10 (conversion rapide avec +)

// vers une chaîne
String(42)          // '42'
(42).toString()     // '42'

// vers un booléen
Boolean(0)          // false
Boolean('')         // false
Boolean('texte')    // true
Boolean(42)         // true
```

> **Astuce —** Les valeurs « falsy » : `0`, `''`, `null`, `undefined`, `NaN`, `false`. Tout le reste est « truthy » !

PARTIE 1 — FONDAMENTAUX

04 Opérateurs

Arithmétiques, comparaison, logiques, affectation

Opérateurs arithmétiques

```
let a = 10, b = 3;
a + b   // 13 — addition
a - b   // 7 — soustraction
a * b   // 30 — multiplication
a / b   // 3.333... — division
a % b   // 1 — modulo (reste de la division)
a ** b  // 1000 — puissance

// incrément / décrément
a++;    // 11 (post-incrément)
++a;    // 12 (pré-incrément)
a--;    // 11 (post-décrément)
```

Opérateurs de comparaison

```
5 == '5'   // true  — égalité faible (conversion de type)
5 === '5'  // false — égalité stricte (type ET valeur)
5 != 6     // true  — différent (faible)
5 !== '5'  // true  — différent (strict)
10 > 5     // true
10 < 5     // false
10 >= 10   // true
10 <= 9    // false
```

Attention — Utilise toujours `===` et `!==` (stricts). `==` peut donner des résultats surprenants à cause des conversions automatiques.

Opérateurs logiques

```
// && — ET (les deux doivent être vrais)
true && true   // true
true && false  // false

// || — OU (au moins un vrai)
false || true  // true

// ! — NON (inverse le booléen)
!true          // false

// ?? — Nullish coalescing (valeur par défaut si null/undefined)
let nom = null ?? 'Anonyme'; // 'Anonyme'
let age = 0 ?? 18;           // 0 (car 0 n'est pas null)
```

Opérateurs d'affectation

```
let x = 10;
x += 5;   // 15
x -= 3;   // 12
x *= 2;   // 24
x /= 4;   // 6
x **= 2;  // 36
x %= 5;   // 1
```

PARTIE 1 — FONDAMENTAUX

05 Conditions

if/else, switch, opérateur ternaire

if / else if / else

```
let note = 75;

if (note >= 90) {
  console.log('Excellent !');
} else if (note >= 70) {
  console.log('Bien !'); // sera affiché
} else if (note >= 50) {
  console.log('Passable');
} else {
  console.log('Insuffisant');
}
```

Opérateur ternaire

Raccourci pour les conditions simples : `condition ? valeur_si_vrai : valeur_si_faux`

```
let age = 20;

// version longue
let statut;
if (age >= 18) { statut = 'majeur'; } else { statut = 'mineur'; }

// version ternaire (équivalente)
let statut2 = age >= 18 ? 'majeur' : 'mineur';
console.log(statut2); // 'majeur'
```

switch / case

Utile pour comparer une valeur à plusieurs possibilités précises.

```
let jour = 'Lundi';

switch (jour) {
  case 'Lundi':
  case 'Mardi':
    console.log('Début de semaine');
    break; // sans break, l'exécution continue au cas suivant !
  case 'Vendredi':
    console.log('Fin de semaine !');
    break;
  default:
    console.log('Jour ordinaire');
}
```

Attention — N'oublie jamais le `break` dans chaque `case`. Sans lui, l'exécution « tombe » dans le cas suivant (fall-through), ce qui est rarement voulu.

PARTIE 1 — FONDAMENTAUX

06 Boucles

for, while, do...while, for...of, for...in

Boucle for classique

```
// initialisation ; condition ; itération
for (let i = 1; i <= 5; i++) {
  console.log(`Tour numéro ${i}`);
}

// compter à rebours
for (let i = 5; i >= 1; i--) {
  console.log(i); // 5, 4, 3, 2, 1
}
```

Boucle while et do...while

`while` répète un bloc tant que la condition est vraie — utile quand on ne connaît pas d'avance le nombre d'itérations. `do...while` s'exécute au moins une fois, même si la condition est fausse dès le départ.

```
let compteur = 0;
while (compteur < 5) {
  console.log('compteur : ' + compteur);
  compteur++; // sans ça → boucle infinie !
}

let x = 10;
do {
  console.log(x); // s'exécute même si x >= 10
  x++;
} while (x < 5);
```

for...of et for...in

```
// for...of — parcourir les VALEURS d'un tableau
const fruits = ['pomme', 'banane', 'orange'];
for (const fruit of fruits) {
  console.log(fruit); // pomme, banane, orange
}

// for...in — parcourir les CLÉS d'un objet
const personne = { nom: 'Ali', age: 25, ville: 'Dakar' };
for (const cle in personne) {
  console.log(`${cle} : ${personne[cle]}`);
}
```

break et continue

```
// break — stopper la boucle
for (let i = 0; i < 10; i++) {
  if (i === 5) break; // arrête à 5
  console.log(i); // 0, 1, 2, 3, 4
}

// continue — passer à l'itération suivante
for (let i = 0; i < 6; i++) {
  if (i === 3) continue; // saute 3
  console.log(i); // 0, 1, 2, 4, 5
}
```

PARTIE 1 — FONDAMENTAUX

07 Fonctions

Déclaration, paramètres, fonctions fléchées, this lexical

Déclarer et appeler une fonction

```
function direBonjour(prenom) {
  console.log(`Bonjour, ${prenom} !`);
}
direBonjour('Fatou'); // Bonjour, Fatou !

// avec valeur de retour
function additionner(a, b) {
  return a + b;
}
let resultat = additionner(5, 3); // 8
```

Paramètres par défaut

```
function saluer(prenom = 'Ami', lang = 'fr') {
  if (lang === 'fr') return `Bonjour, ${prenom} !`;
  if (lang === 'en') return `Hello, ${prenom}!`;
}
saluer();               // Bonjour, Ami !
saluer('Lena', 'en');   // Hello, Lena!
```

ES6+ Fonctions fléchées (arrow functions)

```
// fonction classique
function carre(n) { return n * n; }

// arrow function équivalente
const carre = (n) => n * n;

// un seul paramètre — parenthèses optionnelles
const double = n => n * 2;

// plusieurs lignes — accolades + return
const calculer = (a, b) => {
  const somme = a + b;
  return somme * 2;
};

// aucun paramètre — parenthèses obligatoires
const maintenant = () => new Date();

// retourner un objet littéral — parenthèses obligatoires
const creerUser = (nom, age) => ({ nom, age, actif: true });
```

this : la grande différence

Une fonction classique crée son propre `this` au moment de l'appel. Une fonction fléchée n'a pas son propre `this` : elle capture celui de son scope de définition (*this lexical*). Cela évite le besoin de

`.bind(this)` ou de `const self = this`.

```
class Timer {
  constructor() { this.secondes = 0; }

  // problème avec une function classique
  demarrerAvecFunction() {
    setInterval(function() {
      this.secondes++; // 'this' est undefined ou window !
    }, 1000);
  }

  // solution avec une fonction fléchée
  demarrer() {
    setInterval(() => {
      this.secondes++; // 'this' = l'instance Timer ✓
    }, 1000);
  }
}
```

Attention — Ne pas utiliser les fonctions fléchées comme méthodes d'objet, comme constructeurs (`new`), ni avec `arguments` (non disponible). Elles n'ont pas non plus de prototype.

Fonctions comme valeurs (callbacks)

```
const multiplier = function(a, b) { return a * b; };

function appliquer(fn, x, y) { return fn(x, y); }

appliquer(multiplier, 4, 5);           // 20
appliquer((a, b) => a + b, 4, 5);      // 9
```

À retenir — En JS, les fonctions sont des « first-class citizens » : on peut les stocker dans des variables, les passer en argument, et les retourner depuis d'autres fonctions.

PARTIE 1 — FONDAMENTAUX

08 Tableaux (Arrays)

Créer, modifier, parcourir et transformer des listes

Créer et accéder à un tableau

```
const fruits = ['pomme', 'banane', 'orange'];

console.log(fruits[0]);        // 'pomme' (index commence à 0)
console.log(fruits[2]);        // 'orange'
console.log(fruits.length);    // 3

console.log(fruits[fruits.length - 1]); // 'orange'
console.log(fruits.at(-1));             // 'orange' (ES2022)
```

Méthodes essentielles

```
const arr = [1, 2, 3];
arr.push(4);       // [1,2,3,4] — ajoute à la fin
arr.pop();         // [1,2,3] — retire le dernier
arr.unshift(0);    // [0,1,2,3] — ajoute au début
arr.shift();       // [1,2,3] — retire le premier
arr.includes(2);   // true
arr.indexOf(3);    // 2
arr.slice(0, 2);   // [1, 2] (extrait sans modifier)
arr.join(' - ');   // '1 - 2 - 3'
```

Méthodes fonctionnelles (map, filter, reduce…)

Ces méthodes ne modifient jamais le tableau original : elles retournent toujours un nouveau tableau (ou une valeur).

```
const notes = [12, 8, 15, 5, 18];

// map — transformer chaque élément
const doubled = notes.map(n => n * 2);          // [24, 16, 30, 10, 36]

// filter — garder selon une condition
const recus = notes.filter(n => n >= 10);       // [12, 15, 18]

// find — premier élément qui correspond
const top = notes.find(n => n >= 16);           // 18

// reduce — accumuler en une seule valeur
const total = notes.reduce((acc, n) => acc + n, 0); // 58

// forEach — exécuter pour chaque élément
notes.forEach((n, i) => console.log(`Note ${i + 1}: ${n}`));
```

Astuce — `map`, `filter` et `reduce` ne modifient jamais le tableau original. Elles retournent toujours un nouveau tableau (ou une valeur).

PARTIE 1 — FONDAMENTAUX

09 Objets

Clé-valeur, méthodes, this, déstructuration de base

Créer et accéder à un objet

```
const personne = { prenom: 'Aminata', nom: 'Diallo', age: 28, ville: 'Dakar' };

console.log(personne.prenom);     // notation point — 'Aminata'
console.log(personne['ville']);   // notation crochets (clé dynamique) — 'Dakar'

personne.age = 29;                // modifier
personne.email = 'aminata@mail.com'; // ajouter
delete personne.ville;            // supprimer
```

Méthodes dans un objet

```
const voiture = {
  marque: 'Peugeot',
  vitesse: 0,
  accelerer(km) {
    this.vitesse += km;
    console.log(`Vitesse : ${this.vitesse} km/h`);
  },
  freiner() { this.vitesse = 0; console.log('Arrêt complet'); }
};

voiture.accelerer(50); // Vitesse : 50 km/h
voiture.accelerer(30); // Vitesse : 80 km/h
```

Déstructuration de base

La déstructuration permet d'extraire rapidement des propriétés d'un objet. Le chapitre 11 (Partie 2) en présente la version complète, avec les tableaux, les valeurs imbriquées et l'opérateur rest.

```
const utilisateur = { id: 1, nom: 'Samba', email: 'samba@mail.com', role: 'admin' };

const { nom, email, role = 'user' } = utilisateur;
console.log(nom);   // 'Samba'

// renommer lors de la déstructuration
const { nom: prenom, id: userId } = utilisateur;

// spread operator — copier / fusionner des objets
const copie = { ...utilisateur };
const majRole = { ...utilisateur, role: 'superadmin' };
```

Méthodes utiles sur les objets

```
const scores = { Alice: 90, Bob: 75, Chloe: 88 };

Object.keys(scores);    // ['Alice', 'Bob', 'Chloe']
Object.values(scores);  // [90, 75, 88]
Object.entries(scores); // [['Alice',90], ['Bob',75], ['Chloe',88]]

for (const [nom, score] of Object.entries(scores)) {
  console.log(`${nom} a eu ${score}/100`);
}
```

Fin de la Partie 1 — Tu maîtrises maintenant les fondamentaux de JavaScript. La Partie 2 explore les fonctionnalités modernes apportées par ES6 (2015) et les versions suivantes.

• PARTIE 2

JavaScript ES6+

ECMAScript 2015 a transformé le langage : nouvelle syntaxe, classes, modules, gestion native de l'asynchrone et nouvelles structures de données. Voici les 8 chapitres qui couvrent l'essentiel du JavaScript moderne.

8	2015	ES2015→2022
chapitres	année ES6	couverture

PARTIE 2 — JAVASCRIPT ES6+

10 Template literals

Interpolation, multi-lignes et tagged templates

Chaînes de caractères délimitées par des **backticks** (`` ` ``), permettant l'interpolation d'expressions JavaScript et les chaînes multi-lignes natives.

Interpolation et expressions

```
const prenom = 'Alice', age = 30, prix = 9.99, qte = 3;

// interpolation simple
const msg = `Bonjour, ${prenom} ! Tu as ${age} ans.`;

// expressions complexes dans ${}
const total = `Total : ${(prix * qte).toFixed(2)} EUR`;
const label = `Statut : ${age >= 18 ? 'majeur' : 'mineur'}`;
const appel = `Résultat : ${Math.max(1, 2, 3)}`;

// multi-lignes (les sauts de ligne sont conservés)
const html = `
  <article>
    <h2>${prenom}</h2>
  </article>
`;
```

Tagged templates (avancé)

Un template peut être précédé d'une fonction (tag) qui reçoit séparément les parties statiques et les valeurs interpolées. Utile pour la sanitisation, l'i18n ou les requêtes sécurisées.

```
// tag = fonction(strings, ...valeurs)
function securiser(strings, ...valeurs) {
  return strings.reduce((acc, str, i) => {
    const val = valeurs[i] ? echapper(String(valeurs[i])) : '';
    return acc + str + val;
  }, '');
}

const input = '<script>alert("xss")</script>';
const safe = securiser`Valeur reçue : ${input}`;
// la valeur est échappée avant insertion — usage réel : css-in-js, GraphQL, SQL
```

Astuce — Les template literals supportent l'imbrication (un template dans un `${ }` d'un autre), mais c'est rarement lisible. Préfère des variables intermédiaires.

PARTIE 2 — JAVASCRIPT ES6+

11 Destructuration avancée

Tableaux, objets, valeurs par défaut, imbrication

La destructuration extrait des valeurs de tableaux ou des propriétés d'objets en une seule instruction, avec alias et valeurs par défaut.

Destructuration de tableaux

```
// extraction positionnelle
const [a, b, c] = [1, 2, 3]; // a=1, b=2, c=3

// ignorer des éléments avec des virgules
const [, second, , quatrieme] = [10, 20, 30, 40]; // second=20

// valeur par défaut (si undefined)
const [x = 0, y = 0] = [5]; // x=5, y=0

// swap de variables (sans variable temporaire)
let p = 1, q = 2;
[p, q] = [q, p]; // p=2, q=1

// récupérer le reste
const [tete, ...queue] = [1, 2, 3, 4]; // tete=1, queue=[2,3,4]
```

Destructuration d'objets

```
const user = { nom: 'Alice', age: 30, ville: 'Paris' };

const { nom, age } = user;                      // extraction par nom
const { nom: prenom, age: annees } = user;      // alias (renommage)
const { role = 'visiteur', nom: n2 } = user;    // valeur par défaut
const { nom: n3, ...reste } = user;              // reste de l'objet
// reste = { age: 30, ville: 'Paris' }

// objets imbriqués
const { adresse: { rue, cp } } = {
  adresse: { rue: 'Rue de Rivoli', cp: '75001' }
};

// dans les paramètres de fonction
function afficherUser({ nom, age = 0, role = 'user' }) {
  console.log(`${nom} (${age}) — ${role}`);
}
afficherUser(user); // Alice (30) — user
```

À retenir — La déstructuration ne modifie pas l'objet source — elle crée de nouvelles liaisons. Les valeurs par défaut ne s'appliquent que si la valeur est `undefined` (pas `null` ou `0`).

PARTIE 2 — JAVASCRIPT ES6+

12 Spread & Rest

L'opérateur ... pour étaler ou regrouper des valeurs

L'opérateur `...` a deux rôles opposés selon le contexte : **spread** étale un itérable, **rest** regroupe des valeurs en tableau.

Spread — étaler des valeurs

```
// tableaux
const a = [1, 2, 3], b = [4, 5, 6];
const fusion = [...a, ...b];      // [1,2,3,4,5,6]
const copie = [...a];              // copie superficielle
const ajout = [0, ...a, 4];        // [0,1,2,3,4]

// passer un tableau comme arguments
Math.max(...[3, 1, 4, 1, 5]);      // 5

// objets (ES2018)
const base = { theme: 'dark', lang: 'fr' };
const config = { ...base, lang: 'en', debug: true };
// { theme: 'dark', lang: 'en', debug: true } — lang écrasé par la valeur suivante

// convertir un Set en tableau dédupliqué
const unique = [...new Set([1, 2, 2, 3])]; // [1, 2, 3]
```

Rest — regrouper des valeurs

```
// dans les paramètres de fonction
function somme(premier, second, ...reste) {
  return reste.reduce((acc, n) => acc + n, premier + second);
}
somme(1, 2, 3, 4, 5); // 15

// paramètre rest seul (remplace arguments)
const log = (...args) => console.log(...args);

// dans la destructuration de tableau
const [head, ...tail] = [10, 20, 30, 40]; // head=10, tail=[20,30,40]

// dans la destructuration d'objet
const { a: aVal, ...autres } = { a: 1, b: 2, c: 3 }; // aVal=1, autres={b:2,c:3}
```

Attention — Le spread copie en surface (shallow copy) : les objets imbriqués restent des références partagées. Pour une copie profonde, utilise `structuredClone(obj)` ou `JSON.parse(JSON.stringify(obj))`.

PARTIE 2 — JAVASCRIPT ES6+

13 Classes

POO, héritage, getters/setters, champs privés

Les classes ES6 sont du sucre syntaxique sur la chaîne de prototypes JavaScript : elles rendent la programmation orientée objet plus lisible et intuitive.

Déclaration et instanciation

```
class Animal {
  constructor(nom, age) {       // appelé avec new Animal(...)
    this.nom = nom;
    this.age = age;
  }
  presenter() {                  // méthode d'instance
    return `Je suis ${this.nom}, j'ai ${this.age} ans.`;
  }
  static creer(nom, age) {       // méthode statique — sur la classe
    return new Animal(nom, age);
  }
}

const chat = new Animal('Mimi', 3);
console.log(chat.presenter()); // Je suis Mimi, j'ai 3 ans.
```

Héritage avec extends et super

```
class Chien extends Animal {
  constructor(nom, age, race) {
    super(nom, age);            // OBLIGATOIRE avant tout this
    this.race = race;
  }
  presenter() {                  // surcharge (override)
    return `${super.presenter()} Je suis un ${this.race}.`;
  }
  aboyer() { return `${this.nom} dit : Woof !`; }
}

const rex = new Chien('Rex', 5, 'Labrador');
rex.presenter();           // Je suis Rex, j'ai 5 ans. Je suis un Labrador.
rex instanceof Chien;      // true
rex instanceof Animal;     // true
```

Getters, setters et champs privés

```
class Compte {
  #solde = 0;            // champ privé (ES2022)
  #historique = [];

  constructor(proprietaire) { this.proprietaire = proprietaire; }

  get solde() { return this.#solde; }          // accès comme une propriété
  set solde(montant) {                           // validation avant affectation
    if (montant < 0) throw new RangeError('Solde négatif interdit');
    this.#solde = montant;
  }
  deposer(montant) {
    this.#solde += montant;
    this.#historique.push({ type: 'dépôt', montant });
  }
  get historique() { return [...this.#historique]; } // copie protège l'original
}

const c = new Compte('Alice');
c.deposer(100);
console.log(c.solde); // 100
// c.#solde // SyntaxError — accès privé interdit
```

> **Attention —** Les classes ne sont pas hoistées : elles doivent être déclarées avant leur première utilisation, contrairement aux fonctions classiques.

PARTIE 2 — JAVASCRIPT ES6+

14 Modules ES6

import / export — organiser le code en fichiers indépendants

Système natif d'import/export. Chaque module a son propre scope — rien n'est exposé par défaut.

Export

```
// math.js
export const PI = 3.14159;          // export nommé — autant qu'on veut par fichier
export function carre(n) { return n * n; }
export const cube = n => n ** 3;

export class Vecteur {               // export d'une classe
  constructor(x, y) { this.x = x; this.y = y; }
  longueur() { return Math.sqrt(this.x**2 + this.y**2); }
}

export default function addition(a, b) { return a + b; } // un seul export default par fichier
```

Import

```
import { PI, carre, Vecteur } from './math.js';        // import nommé
import { carre as c2, cube as c3 } from './math.js';   // avec alias
import addition from './math.js';                       // import par défaut
import addition, { PI, carre } from './math.js';        // défaut + nommés
import * as Math2 from './math.js';                     // namespace complet
Math2.carre(4);          // 16
Math2.default(2, 3);     // 5 — l'export default

// import dynamique (asynchrone, ES2020)
const { carre } = await import('./math.js');
```

Dans le HTML — `<script type="module" src="app.js"></script>`. Les modules s'exécutent en mode strict automatiquement, sont différés (defer) par défaut, et ne polluent pas le scope global.

PARTIE 2 — JAVASCRIPT ES6+

15 Promises & async/await

Gérer l'asynchrone proprement

Les Promises représentent une valeur disponible maintenant, dans le futur, ou jamais. `async/await` (ES2017) permet d'écrire du code asynchrone avec la lisibilité du code synchrone.

Créer et chaîner des Promises

```
const fetchData = () => new Promise((resolve, reject) => {
  setTimeout(() => {
    const ok = Math.random() > 0.3;
    if (ok) resolve({ id: 1, nom: 'Alice' });
    else reject(new Error('Connexion échouée'));
  }, 1000);
});

fetchData()
  .then(data => { console.log('Données :', data); return data.id; })
  .then(id => console.log('ID :', id))
  .catch(err => console.error('Erreur :', err.message))
  .finally(() => console.log('Requête terminée'));
```

Méthodes statiques de Promise

```
// Promise.all — toutes en parallèle (échoue si une échoue)
const [user, posts] = await Promise.all([
  fetch('/api/user').then(r => r.json()),
  fetch('/api/posts').then(r => r.json()),
]);

// Promise.allSettled — toutes, succès ou échec
const resultats = await Promise.allSettled([p1, p2, p3]);
resultats.forEach(r => {
  if (r.status === 'fulfilled') console.log(r.value);
  else console.error(r.reason);
});

// Promise.race — la plus rapide gagne
const premiere = await Promise.race([lent, rapide]);

// Promise.any — la première réussie (ES2021)
const firstSuccess = await Promise.any([p1, p2, p3]);
```

async / await

```
async function chargerUser(id) {       // retourne toujours une Promise
  try {
    const response = await fetch(`/api/users/${id}`); // suspend jusqu'à résolution
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return await response.json();
  } catch (err) {
    console.error('Erreur lors du chargement :', err.message);
    throw err; // re-lancer si le caller doit gérer
  }
}

// parallélisme avec await
async function chargerTout() {
  const [user, config] = await Promise.all([
    getData('/api/user'),
    getData('/api/config'),
  ]);
  return { user, config };
}
```

À retenir — `await` ne bloque que la fonction async courante, pas le thread. Pour des opérations séquentielles, enchaîne les `await`. Pour des opérations parallèles, utilise `Promise.all` avec `await`.

PARTIE 2 — JAVASCRIPT ES6+

16 Itérateurs, Map & Set

Nouvelles collections et protocole itérable

Map — dictionnaire à clés quelconques

`Map` est un dictionnaire où les clés peuvent être de n'importe quel type (objets, fonctions, primitives). Contrairement aux objets, l'ordre d'insertion est garanti.

```
const map = new Map();
map.set('string', 'valeur 1');
map.set(42, 'valeur 2');
const obj = {};
map.set(obj, 'valeur 4'); // un objet comme clé !

map.get('string'); // 'valeur 1'
map.has(42);       // true
map.size;           // 3 (après delete d'une clé)

const config = new Map([['host', 'localhost'], ['port', 3000]]);
for (const [cle, valeur] of config) {
  console.log(`${cle} => ${valeur}`);
}
```

Set — collection de valeurs uniques

```
const set = new Set([1, 2, 2, 3, 3, 3]); // {1, 2, 3} — doublons supprimés
set.add(4); set.has(2); set.size; // 4

// astuce : dédupliquer un tableau
const unique = [...new Set([1, 1, 2, 3, 3, 4])]; // [1, 2, 3, 4]

// opérations ensemblistes
const a = new Set([1, 2, 3, 4]), b = new Set([3, 4, 5, 6]);
const union = new Set([...a, ...b]);                          // {1,2,3,4,5,6}
const intersection = new Set([...a].filter(x => b.has(x)));   // {3,4}
```

for...of et le protocole itérable

```
// créer un objet itérable personnalisé
const range = {
  from: 1, to: 5,
  [Symbol.iterator]() {
    let current = this.from;
    const last = this.to;
    return { next() {
      if (current <= last) return { value: current++, done: false };
      return { value: undefined, done: true };
    } };
  }
};
for (const n of range) { console.log(n); } // 1, 2, 3, 4, 5
```

À retenir — `WeakMap` et `WeakSet` sont des variantes dont les clés (objets) sont des références faibles — elles n'empêchent pas le garbage collector. Utiles pour associer des métadonnées sans créer de fuites mémoire.

PARTIE 2 — JAVASCRIPT ES6+

17 Paramètres & objets améliorés

Raccourcis de syntaxe ES6 pour fonctions et objets

Paramètres par défaut avancés

```
// ES6 — valeurs par défaut
function saluer(nom = 'Monde', titre = 'M.') { return `${titre} ${nom}`; }
saluer();              // 'M. Monde'
saluer('Alice', 'Mme'); // 'Mme Alice'

// les expressions sont évaluées à chaque appel
function creerId(base, suffixe = Date.now()) { return `${base}-${suffixe}`; }

// référencer des paramètres précédents
function multiplier(a, b = a * 2) { return a * b; }
multiplier(3); // 3 * 6 = 18
```

Propriétés et méthodes raccourcies

```
const nom = 'Alice', age = 30;
const user = { nom, age }; // shorthand — équivaut à { nom: nom, age: age }

const compteur = {
  valeur: 0,
  incrementer() { this.valeur++; }, // sans 'function'
  reset() { this.valeur = 0; },
};

const cercle = {
  _rayon: 5,
  get rayon() { return this._rayon; },
  set rayon(r) { if (r > 0) this._rayon = r; },
  get surface() { return Math.PI * this._rayon ** 2; },
};
```

Clés dynamiques et Symbol

```
// computed property names
const cle = 'couleur';
const style = {
  [cle]: 'rouge',              // { couleur: 'rouge' }
  [`${cle}Max`]: 100,
};

// Symbol — identifiant unique et immuable
const ID = Symbol('id');
const entite = { [ID]: 42, nom: 'Alice' };
entite[ID]; // 42 — n'apparaît pas dans for...in ni JSON.stringify

// méthodes utiles sur Object
Object.keys(user); Object.values(user); Object.entries(user);
Object.assign({}, a, b); // fusion (remplacé par le spread)
Object.freeze(user);     // rend immuable
```

Astuce — Préfère le spread `{ ...obj }` à `Object.assign({}, obj)` : plus lisible, même comportement pour la copie superficielle. `Object.assign` modifie son premier argument — attention aux effets de bord.

Récapitulatif général

L'essentiel des 17 chapitres, en un coup d'œil.

Partie 1 — Fondamentaux

Chapitre	Concept clé	À retenir
Variables	`let` , `const`	`const` par défaut, `let` si besoin de modifier
Types	7 primitifs	`typeof null === 'object'` (bug historique)
Opérateurs	`===` vs `==`	Toujours utiliser `===` pour comparer
Conditions	if/else, ternaire	`condition ? vrai : faux`
Boucles	for, while	`for...of` pour les tableaux
Fonctions	arrow functions	`const fn = (x) => x * 2`
Tableaux	map/filter/reduce	Ne modifient pas le tableau original
Objets	`{ clé: valeur }`	Destructuring : `const { a, b } = obj`

Partie 2 — ES6+

Chapitre	Syntaxe clé	À retenir
Template literal	`` `Hello ${name}` ``	Backticks, `${}` pour les expressions
Destructuration	`const {a} = obj`	Alias, valeurs par défaut, rest
Spread / Rest	`[...a, ...b]`	Copie superficielle uniquement
Classes	`class A extends B`	`super()` obligatoire, pas de hoisting
Modules	`import` / `export`	Portée propre, mode strict automatique
Promises	`.then().catch()`	`Promise.all` pour le parallélisme
async/await	`async fn` / `await p`	Syntaxe synchrone, toujours avec try/catch
Map / Set	`new Map()` / `new Set()`	Clés de tout type ; valeurs uniques

Bonne pratique générale — Écris du code ES6+ avec `const` et `let`, préfère les fonctions fléchées pour les callbacks, les classes pour la POO, les modules pour l'organisation, et `async/await` pour l'asynchrone.

Pour aller plus loin

Félicitations ! Tu as couvert les fondamentaux et les fonctionnalités modernes de JavaScript. Voici les prochaines étapes pour continuer à progresser.

DOM & Events
Manipuler le HTML dynamiquement depuis JS

TypeScript
Ajouter un typage statique à JavaScript

Node.js & npm
JS côté serveur et gestion de paquets

APIs & Fetch
Communiquer avec des serveurs et des APIs REST

Frameworks modernes
React, Vue ou Svelte pour construire des interfaces

Tests
Jest ou Vitest pour fiabiliser ton code

Conseil — La meilleure façon de progresser reste de coder : reprends chaque exemple de ce cours, modifie-le, casse-le, comprends pourquoi il casse. C'est ainsi qu'on retient vraiment un langage.